DESARROLLADOR DE APLICACIONES

Técnico Superior en Desarrollo de Aplicaciones Multiplataforma

0486 MP Acceso a Datos
UD2 Manejo de conectores

Índice

01. Introducción

02. El desfase objeto-relacional

03. Protocolos de acceso a BBDD: conectores

3.1. Acceso a bases de datos mediante un conector JDBC

3.2. Pool de conexiones

04. Ejecución de sentencias DDL

05. Ejecución de sentencias DML

06. Ejecución de consultas

6.1. Clase Statement

6.2. Clase PreparedStatement

6.3. Clase CallableStatement

07. Gestión de transacciones

08. Bibliografía

01. Introducción

Un conector es conjunto de clases encargadas de implementar la interfaz de programación de aplicaciones (API) y facilitar, con ello, el acceso a una base de datos.

Para poder conectarse a una base de datos y lanzar consultas, una aplicación siempre necesita tener un conector asociado. Desde los lenguajes propios de los sistemas gestores de bases de datos se pueden gestionar los datos mediante lenguajes de consulta y manipulación propios de esos sistemas (tal como vimos en bases de datos en 1º), pero cuando se quiere acceder a los datos desde lenguajes de programación de una misma manera con independencia del sistema gestor que contenga los datos, entonces es necesario utilizar conectores que faciliten estas operaciones.

Por lo tanto, los conectores dan al programador una manera homogénea de acceder a cualquier sistema gestor.

02. El desfase objeto-relacional

El problema del desfase objeto-relacional consiste en la diferencia de aspectos que existen entre la programación orientada a objetos, con la que se desarrollan aplicaciones, y la base de datos, con las que se almacena la información.

Estos aspectos se pueden presentar relacionados en las siguientes situaciones:

- **Cuando se realizan actividades de programación**, donde el programador debe conocer el lenguaje de POO y el lenguaje de acceso a datos.
- **Cuando se especifican los tipos de datos**. En las bases de datos relacionales siempre hay restricciones en cuanto a los tipos de datos que se pueden usar, suelen ser tipos simple, mientras que en la POO se utilizan tipos de datos complejos.
- **En el proceso de elaboración del software se realiza una traducción del modelo OO al modelo (E/R),** puesto que el primero maneja objetos y el segundo maneja tablas y tuplas o filas, lo que implica que el desarrollador tenga que diseñar dos diagramas diferentes para el diseño de la aplicación.

En definitiva, la discrepancia objeto-relacional surge porque:

- En el modelo relacional se trata con relaciones y conjuntos debido a su naturaleza matemática.
- En el modelo de POO se trabaja con objetos y las asociaciones entre ellos.

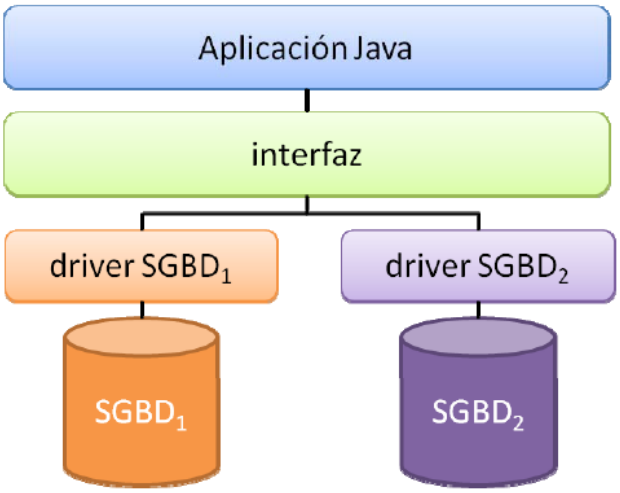
Todo este conjunto de dificultades técnicas que aparecen cuando una base de datos relacional se usa juntamente con un programa escrito con lenguajes de programación orientada a objetos se denomina **desfase objeto-relacional**.

03. Protocolos de acceso a BBDD: conectores

Muchos servidores de bases de datos utilizan protocolos de comunicación específicos que facilitan el acceso a los mismos, lo que obliga a aprender un lenguaje nuevo para trabajar con cada uno de ellos. Sin embargo, es posible reducir esa diversidad de protocolos mediante alguna interfaz de alto nivel que ofrezca al programador una serie de métodos para acceder a la base de datos.

Estas interfaces de alto nivel ofrecen facilidades para:

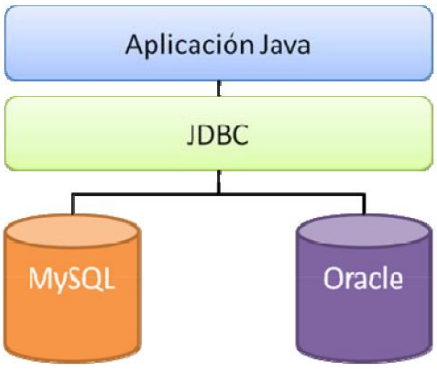
- Establecer una conexión a una base de datos.
- Ejecutar consultas sobre una base de datos.
- Procesar los resultados de las consultas realizadas.



Al conjunto de clases encargadas de implementar la interfaz de programación de aplicaciones (API) y facilitar, con ello, el acceso a una base de datos es a lo que denominamos **conector o driver**.

Un ejemplo de conector muy extendido es el **conector JDBC**. Este conector es una capa software intermedia situada entre los programas Java y los sistemas de gestión de bases de datos relacionales que utilizan SQL.

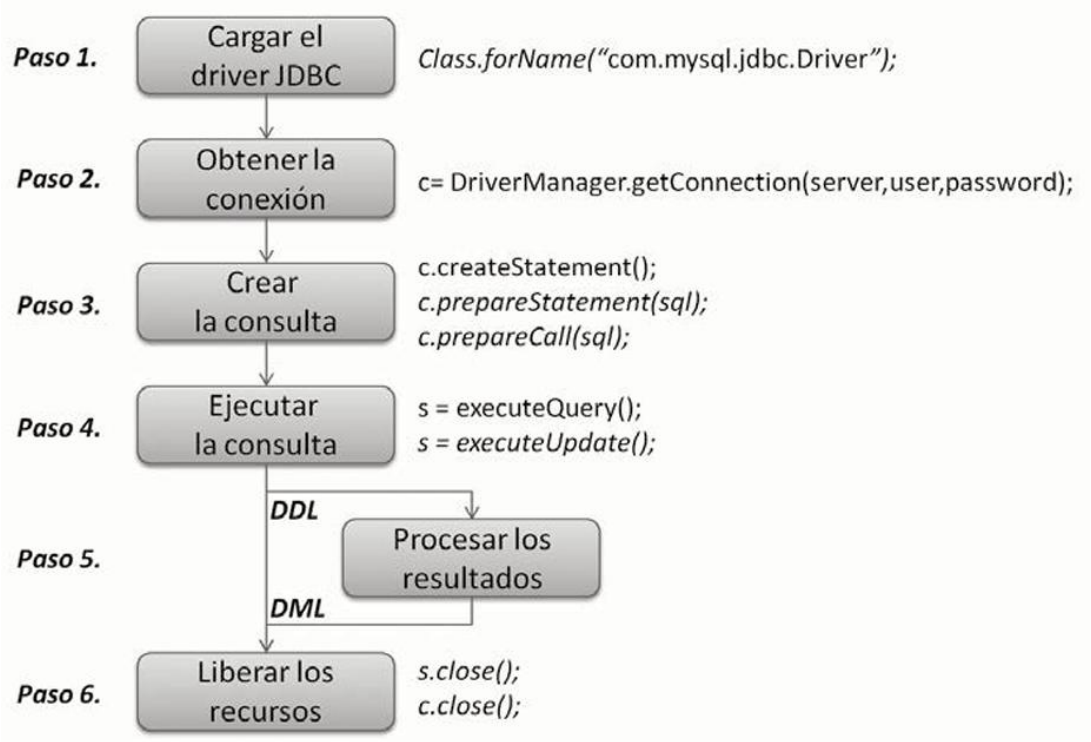
Con el conector JDBC no hay que escribir un programa para acceder, por ejemplo, a una base de datos Access y otro programa distinto para acceder a una base de datos Oracle, etc.



Otro ejemplo de conector es el conector de **Microsoft ODBC**.

3.1. Acceso a bases de datos mediante un conector JDBC

Para acceder a una base de datos MySQL con JDBC se deben seguir los pasos que se muestran resumidamente en la siguiente imagen.



Como vemos en este esquema, lo **primero que se necesita para conectarnos a una base de datos es cargar el driver JDBC, por lo tanto, necesitamos un objeto conector**. Ese conector es el que sabe cómo interactuar con la base de datos.

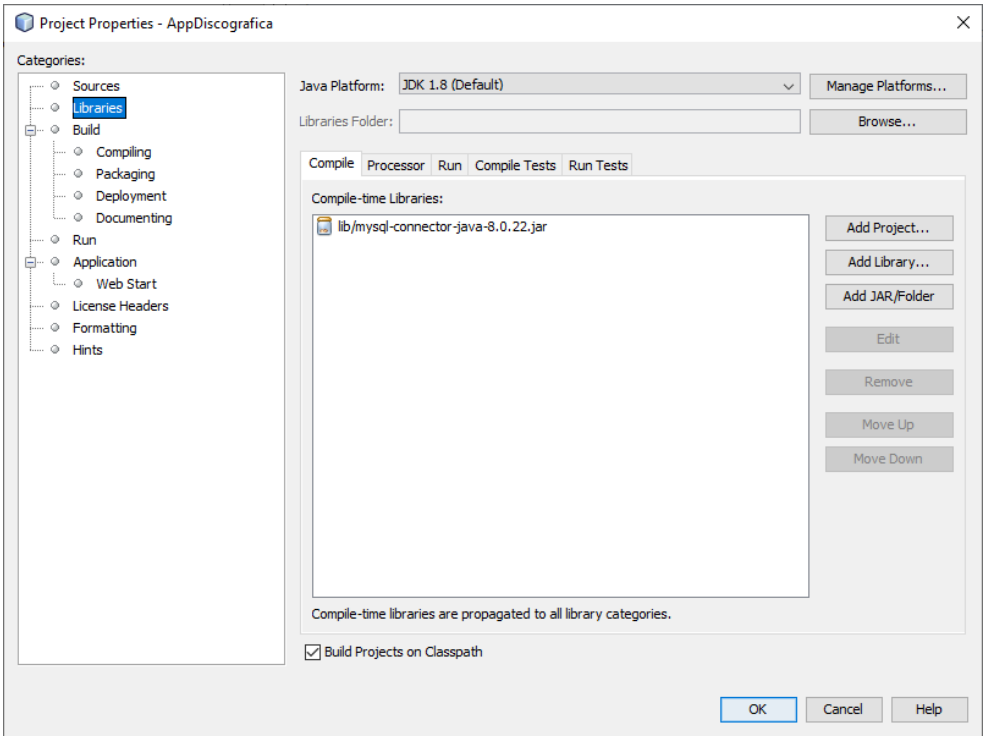
El lenguaje Java no viene con todos los conectores de todas las posibles bases de datos del mercado. Se debe recurrir a Internet para obtener el conector que se necesite en cada caso.

En nuestros ejemplos, necesitaremos el conector de MySQL. (http://dev.mysql.com/downloads/connector/j)

Una vez descargado el fichero **mysqlconnector-java-8.0.xx.zip**, se descomprime y se localiza el fichero mysql-connector-java-8.0.xx.jar, incluido en el fichero .zip.

Para incluir el fichero mysql-connector-java-8.0.xx.jar en cada proyecto se deberán seguir los siguientes pasos:

- En la carpeta raíz del proyecto, crear la carpeta **/lib**.
- Copiar el fichero mysql-connector-java-8.0.xx.jar en la carpeta /lib que se acaba de crear.
- Desde NetBeans pulsar con el botón derecho del ratón en el nombre del proyecto y seleccionar la opción de menú **propiedades**.
- En el árbol lateral pulsar en **Libraries**
- En el botón de la izquierda pulsar en **Add JAR/Folder**.
- Seleccionar el fichero **mysql-connector-java-8.0.xx.jar** que se encuentra en la carpeta /lib del proyecto y pulsar **Abrir**.
- Pulsar **OK**.



Puedes ver estos pasos en el vídeo 1 de este tema, cargar el conector JDBC en nuestro proyecto NetBeans.

Una vez cargado el driver JDBC, el segundo paso que tenemos que realizar según el esquema de la página 6 es **obtener la conexión**. Esto lo veremos con un ejemplo.

El nuestro ejemplo accederemos a una base de datos discográfica creada en MySQL. Esta base de datos tiene una tabla ÁLBUM con tres campos:

- Id, clave primara de tipo numérico.
- Título de tipo VARCHAR(30).
- Autor de tipo VARCHAR(30).

El objetivo es conseguir un objeto del tipo java.sql.Connection a través del método DriverManager.getConecction(string url).

De esta manera el constructor de la clase GestorConexión quedaría de la siguiente manera:

```
public GestorConexion() {

    try {
        String url1 = "jdbc:mysql://localhost:3306/discografica?serverTimezone=UTC";
        String user = "root";
        String password = "root";

        conn1 = DriverManager.getConnection(url1, user, password);

        if (conn1 != null) {
            System.out.println("Conectado a discográfica...");
        }
    } catch (SQLException ex) {
        System.out.println("ERROR en la conexión a la BBDD discográfica");
        ex.printStackTrace();
    }
}
```

Te aconsejo que veas en este punto el vídeo 2 de este tema, obtener la conexión con la base de datos (os avanzo que en este vídeo hay un error que se corrige en el vídeo 3, a ver si descubrís vosotros el error, la imagen con el código que se ve en estos apuntes es correcta).

Después de usar una conexión, ésta debe ser cerrada con el método close() de Connection.

```
public void cerrar_conexion (){
    try {
        conn1.close();
    }catch (SQLException ex) {
        System.out.println("ERROR: cierre de la conexión");
    }
}
```

Cómo programar este método lo puedes ver en el vídeo 3 de este tema, cerrar conexión.

3.2. Pool de conexiones

La manera mostrada en el apartado anterior de obtener una conexión es correcta para aplicaciones sencillas, donde únicamente se establece una conexión con la base de datos. Sin embargo, hay problemas cuando existen varios hilos de ejecución ya que no pueden usar una misma conexión física con la base de datos simultáneamente.

Para este problema tenemos varias posibles soluciones:

- Abrir y cerrar una conexión cada vez que la necesitemos.
- Usar una única conexión y sincronizar el acceso a ella desde los distintos hilos.
- Tener varias conexiones abiertas (pool de conexiones).

Abrir y cerrar una conexión cada vez que la necesitemos.

De esta manera cada hilo de ejecución tendrá su propia conexión. Esta solución no es eficiente, puesto que establecer una conexión real con la base de datos es un proceso costoso y puede hacer que el programa vaya más lento.

Usar una única conexión y sincronizar el acceso a ella desde los distintos hilos.

Solución más eficiente, pero es necesario poner siempre synchronized antes de hacer cualquier transacción con la base de datos. Tiene además la pega que los hilos deben esperar entre ellos.

Tener varias conexiones abiertas (pool de conexiones).

Cuando un hilo necesita una conexión, la pide, y cuando termine, la deje para que pueda ser usada por los demás hilos sin tener que abrir y cerrar conexión cada vez. Si hay conexiones disponibles, un hilo no tiene que esperar a que otro acabe.

```
public PoolConexiones(){

    BasicDataSource bdSource = new BasicDataSource();
    bdSource.setUrl ("jdbc:mysql://localhost:3306/discografica?serverTimezone=UTC");
    bdSource.setUsername("root");
    bdSource.setPassword("root");

    try {
        //DataSource nos reserva una conexión y nos la devuelve
        con = bdSource.getConnection();
        if (con != null) {
            System.out.println("Conexión creada satisfactoriamente");
        } else {
            System.out.println("No se puede crear una nueva conexión");
        }
    } catch (Exception e) {
        System.out.println("Error: " + e.toString());
    }
}
```

Esta aplicación necesita de la librería **commons-dbcp2-2.1.1.jar**, junto con sus dependencias **commons-logging-1.2.jar** y **commons-pool2-2.4.2.jar**, por lo que es necesario seguir el mismo proceso mostrado para el driver de MySQL para utilizarlas: incluir estas tres librerías en la carpeta lib del proyecto, y posteriormente añadirlas a la librería de NetBeans.

Mi objeto BasicDataSource puede ser ampliamente configurado. Uno de los parámetros que puedo configurar es el número máximo de conexiones con la base de datos que deseo que estén abiertas. Esto lo puedo configurar con el método setMaxActive(). De esta forma cada vez que un hilo ejecute el método getConnection() se le proporcionará una de las conexiones disponibles.

Podéis ver explicado el pool de conexiones en el vídeo 4 de este tema.

04. Ejecución de sentencias DDL

Como vimos el curso pasado en el módulo de Bases de datos, el lenguaje de definición de datos (DDL) es la parte de SQL dedicada a la definición de una base de datos. Las principales sentencias asociadas con el lenguaje DDL son:

- **CREATE**: sirve para crear una base de datos o un objeto.
- **ALTER**: sirve para modificar la estructura de una base de datos o de un objeto.
- **DROP**: permite eliminar una base de datos o un objeto.

Para enviar comandos SQL a la base de datos con JDBC se usa un objeto de la **clase Statement**. Este objeto se obtiene a partir de una conexión a la base de datos:

Statement st = conexion.createStatement();

Statement tiene muchos métodos, pero hay dos especialmente interesantes:

- **executeUpdate()**: se usa para sentencias SQL que impliquen modificaciones en la base de datos (INSERT, UPDATE, DELETE, CREATE, ALTER, etc).
- **executeQuery()**: se usa para consultas (SELECT).

Por lo tanto, es el primero de ellos **executeUpdate()**, el que utilizaremos en este apartado para ejecutar sentencias DDL en nuestra aplicación.

Ejemplo: Si queremos añadir una nueva columna a una tabla ya existente, el código que necesitamos ejecutar es:

```
public void annadirColumna(){

    Statement sta;
    try {
        sta = connl.createStatement();
        sta.executeUpdate("ALTER TABLE album ADD anno_publicacion YEAR;");
        sta.close();
    } catch (SQLException ex) {
        System.out.println(ex.toString());
    }

}
```

Ejemplo: Si querremos borrar una tabla existente, ojo porque borra la tabla entera, tanto el contenido como la estructura (no vacía la tabla), el código que necesitamos ejecutar es:

```
public void borrarTabla(){
    Statement sta;
    try {
        sta = connl.createStatement();
        sta.executeUpdate("DROP table album;");
        sta.close();
    } catch (SQLException ex) {
        System.out.println(ex.toString());
    }
}
```

Podéis ver cómo poner estos ejemplos en nuestra aplicación con acceso a discográfica en el vídeo 5 de este tema.

05. Ejecución de sentencias DML

Las sentencias de manipulación de datos (DML) son las utilizadas para insertar, borrar, modificar y consultar los datos que hay en una base de datos. Las principales sentencias DML son:

- **SELECT:** sirve para recuperar información de una base de datos y permite la selección de una o más filas y columnas de una o muchas tablas.
- **INSERT:** se utilizan para agregar registros a una tabla.
- **UPDATE:** permite modificar la información de las tablas.
- **DELETE:** permite eliminar una o más filas de una tabla.

Del mismo modo que ocurría con las sentencias DDL vistas en el punto anterior, las sentencias DML que implican una modificación sobre la base de datos, por lo tanto, para ejecutarlas utilizaremos el método **executeUpdate()** de la clase Statement.

La sentencia SELECT, sin embargo, no utilizará este método, sino executeQuery. En el siguiente apartado veremos cómo realizar consultas sobre la base de datos desde Java.

Por lo tanto, un método que nos permita insertar datos en nuestra BBDD discográfica desde la aplicación Java sería:

```
public void insertarAlbum(){
    Statement sta;
    try {
        sta = conn1.createStatement();
        sta.executeUpdate("INSERT INTO album VALUE (1, 'Greatest Hits', 'Queen');");
        sta.close();
    } catch (SQLException ex) {
        System.out.println(ex.toString());
    }
}
```

Podéis ver cómo este ejemplo en el vídeo 6 de este tema.

06. Ejecución de consultas

La ejecución de consultas sobre bases de datos desde aplicaciones Java descansa en dos tipos de clases disponibles en el API JDBC (**Statement y ResultSet).**

6.1. Clase Statement

Como ya hemos visto en el apartado anterior, la clase Statement nos permite ejecutar las sentencias SQL estáticas. Podemos obtener un objeto de esta clase a través del método createStatement de nuestro objeto Connection (**Connection.createStatement()).**

El método **executeQuery()** de Statement está diseñado para sentencias que devuelvan un único resultado (**ResultSet**), como es el caso de las sentencias SELECT.

```
ResultSet res = sta.executeQuery();
```

Los objetos de la clase **ResultSet** representan la respuesta a las peticiones que se hacen a una base de datos, por lo que esta clase no es más que un conjunto ordenado de filas de una tabla (recordamos del curso pasado que el resultado de una consulta viene dado en forma de tabla).

Asociados a la clase ResultSet existen métodos como **next() y getXXX()** para iterar por las filas y obtener los valores de los campos deseados.

El método next() permite recorrer todas las filas para ir sacando cada uno de los valores devueltos. De este modo, después de invocar al método next(), el resultado recién traído está disponible en el ResulSet.

La forma de recoger los campos es pedirlos con algún método **getXXX**(), de tal manera que si se sabe de qué tipo es el dato, se puede pedir con **getInt(), getString()**, etc., pero si no se sabe o da igual el tipo, bastará con un **getObject(),** que es capaz de traer cualquier tipo de dato.

Por último, al terminar de usar ResultSet y Statement, éstos deben cerrarse, **close(),** para evitar errores inesperados.

El siguiente código muestra un ejemplo de acceso a la base de datos con una consulta SELECT que obtiene todos los álbumes de la base de datos cuyo autor sea "Queen".

```
public void consulta_Statement(){

    Statement sta;

    try {
        sta = connl.createStatement();

        String query = "SELECT * FROM album WHERE autor='Queen';";
        ResultSet rs = sta.executeQuery(query);

        while (rs.next()){
            System.out.println("ID: " + rs.getInt("id") +
                        ", Autor: " + rs.getString("autor") +
                        ", Título: " + rs.getString("titulo")
                    );
        }

        rs.close();
        sta.close();

    } catch (SQLException ex) {
        System.out.println(ex.toString());
    }
}
```

En el vídeo 7 de este tema, puedes ver este ejemplo.

6.2. Clase PreparedStatement

Una variante de la sentencia Statement es la sentencia **PreparedStatement**. Ésta se utiliza para ejecutar las **sentencias SQL precompiladas** y permite que los parámetros de entrada sean establecidos de forma dinámica, ganando eficiencia.

La diferencia principal con respecto a los Statement es que **las consultas pueden tener valores indefinidos que se establecen con el símbolo interrogación (?).** Por lo tanto, en las consultas pondremos tantas interrogaciones como parámetros se quieran usar.

En la imagen siguiente se muestra un método que permite mostrar todos los álbumes del autor que especifique el usuario utilizando una sentencia PreparedStatement.

```
public void consuta_PreparedStatement(String autor){

    String query="SELECT * FROM album WHERE autor= ?;";

    try {
        PreparedStatement pst = conn1.prepareStatement(query);
        pst.setString(1, autor);

        ResultSet rs = pst.executeQuery();

        while (rs.next()){
            System.out.println("ID: " + rs.getInt("id") +
                        ", Autor: " + rs.getString("autor") +
                        ", Título: " + rs.getString("titulo")
                    );
        }

        rs.close();
        pst.close();

    } catch (SQLException ex) {
        System.out.println(ex.toString());
    }
}
```

Como se observa en el código, al crear el preparedStatement se precompila la consulta para dejarla preparada para recibir los valores de los parámetros. De esta manera si el parámetro que se quiere colocar es de tipo Int se usará **setInt(),** pero en nuestro caso el parámetro es de tipo texto por lo que se usa **setString().**

Si nos fijamos en el método **setString(1, autor),** podemos observar que el primer valor (1) indica que la variable autor se asocia con la primera interrogación que se encuentra. Si hubiese más parámetros (?) entonces habría que indicar la posición

que ocupa para que el preparedStatement sepa a cuál asociarle el valor (setString(2,""), setString(3, “”), etc.).

Puedes ver también este ejemplo en el vídeo 8 de este tema.

6.3. Clase CallableStatement

Los objetos de la clase CallableStatement son **sentencias preparedStatement que llaman a un procedimiento almacenado**, es decir, métodos incluidos en la propia base de datos. Aunque no todos los gestores de bases de datos admiten este tipo de procedimientos.

La manera de proceder es igual que con preparedStatement aunque lo que se le da como parámetro es el nombre del procedimiento almacenado junto con los valores de los parámetros del procedimiento puestos como parámetros del Statement.

A continuación, se muestra un ejemplo de llamada para un supuesto procedimiento almacenado llamado DameAlbumes (titulo, autor), que devuelve los álbumes cuyo título y autor coincida con los valores dados.

```
CallableStatement cs = conn1.prepareCall("CALL DameAlbumes(?,?)");
cs.setString(1, “Black%”);
cs.setString(2, “Metallica”);
```

07. Gestión de transacciones

Una transacción en un SGBD es un conjunto de órdenes que se ejecutan de forma indivisible y atómica. Una transacción se inicia cuando se encuentra una primera sentencia DML y finaliza cuando se ejecuta alguna de las siguientes sentencias:

- Un **COMMIT o un ROLLBACK**.
- Una **sentencia DDL**, por ejemplo CREATE.
- Una **sentencia DCL**, por ejemplo GRANT o REVOKE.

Después de que una transacción finaliza, la siguiente sentencia ejecutada automáticamente inicia la siguiente transacción.

En JDBC por omisión cada sentencia SQL se confirma tan pronto se ejecuta, es decir, una conexión funciona **por defecto en modo auto-commit**. Para ejecutar varias sentencias en una misma transacción es preciso deshabilitar el modo auto-commit. Después se podrán ejecutar las instrucciones, y terminar con un COMMIT si todo va bien o un ROLLBACK en otro caso.

En el siguiente ejemplo se puede observar que, si las dos operaciones Insert se ejecutan correctamente, entonces se aplica un commit antes de salir. Sin embargo, si no es así, y salta una excepción, hay que hacer un rollbak(). Es obligatorio poner el rollback dentro de un try-catch. Este ejemplo lo puede ver también en el vídeo 9 de esta unidad.

```
public void insertarConCommit(){

    Statement sta;
    try {
        conn1.setAutoCommit(false);

        sta = conn1.createStatement();

        sta.executeUpdate("INSERT INTO album VALUE (7, 'prueba', 'The Beatles');");
        sta.executeUpdate("INSERT INTO album VALUE (6, 'Abbey Road', 'The Beatles');");

        sta.close();

        conn1.commit();

    } catch (SQLException ex) {
        System.out.println(ex.toString());
        if (conn1!=null){
            try {
                conn1.rollback();
            } catch (SQLException ex1) {
               System.out.println(ex1.toString());
            }
       }
    }
}
```

08. Bibliografía

Córcoles Tendero, J.E. (2013). Acceso a datos. Editorial Ra-Ma.

Ramos Martín, A. y Ramos Martín, MJ. (2016). Acceso a datos. Editorial Garceta.

Cortijo Bon, C.A. (2019). Acceso a datos. Editorial Síntesis.